

The Behavioral Toolbox

Ivan Markovsky

In memory of Jan C. Willems (1937–2012),
who shaped my thinking and whose vision of rethinking systems theory continues to live on.

Contents

Overview

The Behavioral Toolbox is a collection of Matlab functions for analysis and design of dynamical systems using the behavioral approach to systems theory and control. It implements newly emerged nonparametric data-driven methods for linear time-invariant systems. In order to install it, download and unpack the archive file. In order to cite it, please use:

I. Markovsky. The Behavioral Toolbox. In Proc. of Machine Learning Research, volume 242, pages 130–141, 2024.

```
@InProceedings{bt-14dc,  
  Author = {I. Markovsky},  
  Title = {The Behavioral Toolbox},  
  booktitle = {Proc. of Machine Learning Research},  
  volume = {242},  
  pages = {130--141},  
  Year = {2024}  
}
```

Functions

The Behavioral Toolbox supports the following representations:

- `ss` : inputs/state/output,
- `R` : kernel

$$\mathcal{B} = \ker R(\sigma) := \{ w \mid R(\sigma)w = 0 \}, \quad (\text{KER})$$

- `Rp` : shortest-lag kernel,
- `BT` : basis for the finite-horizon behavior

$$\mathcal{B}|_T = \text{image } B_T, \quad \text{where } B_T^\top B_T = I_r, \text{ with } r := \dim \mathcal{B}|_T, \quad (B_T)$$

- `w` : raw data

$$\mathcal{B}|_T = \text{image } \mathcal{H}_T(\mathcal{W}_d). \quad (\text{DDR})$$

The functions of the toolbox that make transitions among the representations have names `x2y` where `x` is the given representation and `y` is the desired one.

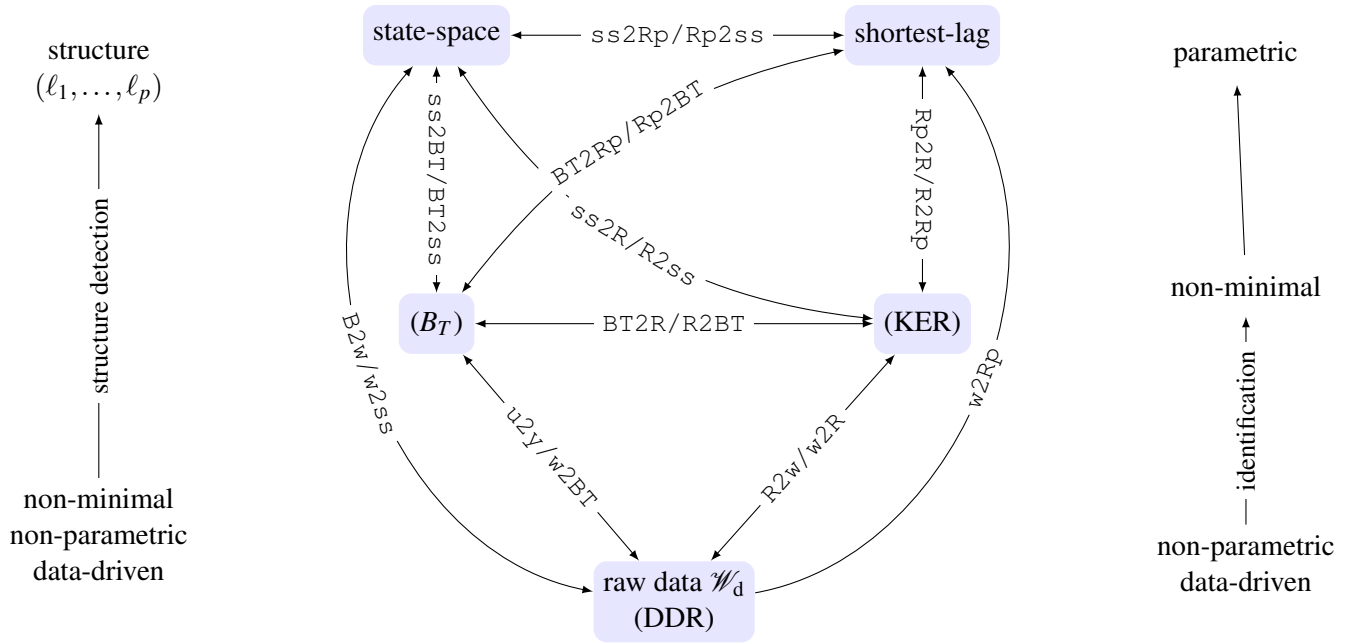


Figure 1: The Behavioral Toolbox supports five LTI system's representations—state-space, shortest-lag kernel, kernel, orthogonal basis for the finite-horizon behavior (B_T), and data-driven (DDR). The state-space and shortest-lag kernel representations are parametric, while (B_T) and (DDR) are non-parametric. The function converting representation x to representation y is named $x2y$. The shortest-lag kernel representation reveals the system's structure $(\ell_1, \dots, \ell_p)$. Going "up" in the diagram from a less structured to a more structured representation requires structure detection.

[width=.9]diagram

In addition to the functions for transformations among representations, the toolbox offers functions for finding input/output partitionings of variables, computation of subbehaviors, analysis, signal processing, open-loop control, identification, and interconnection. A full list of functions is available in pdf.

Examples

[level-1] [level-1]0

In the examples shown below, a trajectory of a discrete-time linear time-invariant system — *the data* — is given and the goal is to find some property of the data-generating system or solve a problem involving the data-generating system. The data is assumed to fully specify the system — a condition that is verifiable from the data and the system's complexity (number of inputs and order).

Finding the number of inputs

The data is a vector time series containing the observed variables of the system. Since these variables are not partitioned into inputs and outputs, which variables are inputs is not known. Also, how many variables are inputs is not known. It is possible however to find the number of inputs from the given data. The function of the toolbox that does this is `w2c`. In addition to finding the number of inputs m , it finds also the order n and the lag ℓ of the underlying data-generating system.

Here is an example:

- specify the simulation parameters—number of variables q , number of inputs m , lag ℓ , order n , and number of data samples T_d

```
q = 3; m = 1; ell = 2; n = 3; Td = 20;
```

- generate a random linear time-invariant system with the specified parameters

```
B = randB(q, [m, ell, n]);
```

- generate a random trajectory of the system with the specified number of samples

```
wd = ss2w(B, Td);
```

- call the w2c function

```
[~, mh, ellh, nh] = w2c(wd);
```

- check if the computed result is correct

```
[m == mh, ell == ellh, n == nh] % -> [true true true]
```

But what about the assumption? A “sufficiently long” random trajectory almost surely fully specifies the system. Quantifying what “sufficiently long” means, however, requires knowledge of m and n . There is no way out of the dilemma, either we have to assume that the data fully specifies the system, or we have to know m and n , in which case we can check the condition.

Exercise: Change the simulation parameters and find out empirically when the data is “sufficiently long”.

Finding an input/output partitioning of the variables

We’ve seen that we can find the number of inputs m from the data. Can we find also *the* input variables u ? No, in general, we can not do this because different sets of variables can act as inputs. The function of the toolbox that finds all possibilities of partitioning the variables into inputs and outputs is BT2IO.

BT2IO (as well as other functions of the toolbox) takes as an input argument a matrix B_T that can be obtained from the data using w2BT. B_T defines a non-parametric representation of the finite-horizon behavior $\mathcal{B}|_T$. The horizon T is a problem dependent. For BT2IO the required horizon is $T = 2\ell + 1$.

Continuing the example above,

- first, we obtain the matrix B_T :

```
T = 2 * ell + 1; BT = w2BT(wd, T, [m ell n]);
```

- then, we call BT2IO:

```
IO = BT2IO(BT, q)
```

Its output

```
IO =
```

1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

is a matrix, the rows of which indicate the possible input/output partitionings, e.g.,

```
ud = wd(:, IO(1, 1:m)); yd = ud = wd(:, IO(:, m+1:end));
```

is the first input/output partitioning.

Exercise: In the example above, all partitionings of the variables are valid input/output partitionings. Create an example where this is not the case.

The data-generating system may be unstable (but it's OK)

In the examples above, we've used the function `randB` for obtaining a random data-generating system B and the function `ss2w` for obtaining a random trajectory of that system.

- `randB` constructs a kernel representation R corresponding to the specified structure (number of variables, number of inputs, lag, and order) with nonzero elements generated as random uniformly distributed in the interval $[0, 1]$. The kernel representation R is then converted to an input/state/output representation B .
- `ss2w` uses the function `lsim` from the Control System Toolbox of Matlab in order to simulate a trajectory of B under random initial conditions and a random input.

The system produced by `randB` may be unstable:

```
[B, R] = randB(q, [m, ell, n]); any(abs(eig(B)) > 1) % -> true
```

When B is unstable, the trajectory obtained from random input and initial conditions is almost surely exponentially diverging

```
T = 300; w = ss2w(B, T); max(w(:)) % -> large
```

Generating a “sufficiently long well behaved” random trajectory from unstable system requires special care (e.g., design of a stabilizing controller).

An alternative way to generate a random trajectory of the system is using the nonparametric representation B_T of the finite-horizon behavior. For a random vector $g \in \mathbb{R}^{mT+n}$, we obtain a random trajectory $w = B_T g$. When $\|g\|$ is bounded (e.g., its elements are uniformly distributed in the interval $[0, 1]$), since by construction the matrix B_T is orthonormal, the resulting trajectory w is also bounded.

```
BT = B2BT(B, T); w = BT * rand(size(BT, 2), 1); max(w) % -> small
```

Thus, using the nonparametric representation B_T of the finite-horizon behavior, standard linear algebra (computation of an orthonormal basis) allows us to obtain “sufficiently long well behaved” trajectories of unstable systems.

Distance of a signal to a system

The distance of a signal w to a system $\mathcal{B}$ has the familiar definition and geometrical interpretation: $\min_{\hat{w} \in \mathcal{B}} \|w - \hat{w}\|$. It's computation is implemented in the function `dist`. As a specification of $\mathcal{B}$, `dist` accepts an `lti` object as well as the finite-horizon representation B_T with horizon T the length of w .

Finding the distance of a signal to a system is useful in particular for checking if the signal is a valid trajectory of the system:

```
dist(wd, B) % -> 0
```

The function `dist` returns also the projection $\hat{w}$ of w in $\mathcal{B}$. The signal $\hat{w}$ is useful because it the best approximation of w by a trajectory of the system. It is called the smoothed signal. Here is an example.

- We add a perturbation on the trajectory and compute its distance to the system:

```
wd_noisy = wd + randn(size(wd)); [d, wdh] = dist(wd_noisy, B);
```

- The distance of the best approximation to the original trajectory is smaller than the distance of the perturbed trajectory to the original trajectory:

```
[norm(wd - wdh) norm(wd - wd_noisy)] / norm(wd)
```

Distance between systems

How to check if two systems are equal? By construction the system

```
Bp = ss2ss (B, rand (n) );
```

is equal to B. However, the Control Systems Toolbox of Matlab does not allow us to infer it by

```
B == Bp % -> Operator '==' is not supported for operands of type 'ss'.
```

Instead, we can check the norm of difference:

```
norm (Bp - B) % -> 0
```

The norm computation, however, is restricted to stable systems.

The function Bdist computes a distance between systems defined as the angle between the corresponding behaviors:

```
Bdist (Bp, B) % -> 0
```

This distance measure is applicable also to stable systems.

Extra examples

The package contains scripts showing the following examples:

script name	example
observer	missing input estimation
smoothing	errors-in-variables smoothing
dd_forecasting	data-driven forecasting
interconnection	interconnection as variables sharing
controllability	distance to uncontrollability
sysid	system identification with predefined data
sysid_missing_data	system identification from data with missing values

[level-1]